

The Pitch

What to say, out loud, tomorrow

15 July 2026

Contents

1 The pitch — what to say tomorrow	1
1.1 Part 0 — Before he sits down (do this in the morning)	1
1.2 Part 1 — The opening (30 seconds)	1
1.3 Part 2 — PART A, the swarm (about 4 minutes)	2
1.4 Part 3 — PART B, decision making (about 5 minutes)	3
1.5 Part 4 — What to say about the papers	6
1.6 Part 5 — If he asks about the equations	7
1.7 Part 6 — Own your weaknesses before he finds them	9
1.8 Part 7 — If something breaks	9
1.9 THE CHEAT CARD — print this, keep it in front of you	9

1 The pitch — what to say tomorrow

Everything below is written to be **said out loud**. Plain words. Where it says “say this”, that is the actual sentence.

Total: about **12 minutes** if he lets you run. He will interrupt. That is fine — every likely interruption has an answer further down.

1.1 Part 0 — Before he sits down (do this in the morning)

```
cd 3
./run.sh rl > /tmp/partB.txt      # 4 minutes. Do NOT run this while he watches.
./run.sh ui                      # leave the browser open on Part A
```

Have open, in this order, ready to switch between:

1. The browser, on **Part A**.
2. results/plots/collective_behaviour.png
3. /tmp/partB.txt
4. results/plots/rl_policy_maps.png
5. results/plots/rl_model_mismatch.png
6. report/main.pdf — your fallback for any number you forget.

Your name and email are already in report/main.tex and the PDF is built. Nothing left to fill in.

1.2 Part 1 — The opening (30 seconds)

Say this first. It makes the two halves look like one assignment instead of two homeworks stapled together.

"The assignment had two parts, so I built two problems, and I put them both in the same University of Dhaka hall. Part A places the hall's Wi-Fi using a swarm. Part B decides when to run the hall's water pump, using an MDP. Same building, two completely different kinds of hard."

Then stop talking and let him steer. If he says nothing, go to Part A.

1.3 Part 2 — PART A, the swarm (about 4 minutes)

1.3.1 2.1 Set the problem up (30 seconds, no maths)

"The hall floor has 40 rooms and some concrete walls. I can afford 3 access points. Where do I bolt them to the ceiling?"

Then give him the one fact that explains everything:

"An access point reaches about 25 metres through open air. Through one concrete wall, it reaches 14. So a wall costs you 11 metres of range. That's why you can't just put them in the middle."

1.3.2 2.2 Why a swarm and not calculus (30 seconds — this is the key idea)

This is the sentence that shows you understand the problem rather than just coding it.

"The reason this needs a swarm is the walls. The number of walls between a room and an access point is a whole number — zero walls, or one, or two. It's never one and a half. So when I slide an access point smoothly across the floor, the signal in some room doesn't fade smoothly — it drops 8 decibels in one jump the moment the line of sight crosses a wall.

The objective has a step in it. You can't do calculus on a step. There's no slope to roll down, so gradient descent has nothing to work with. That's why I reached for a population method."

1.3.3 2.3 Run it live (30 seconds)

In the UI, hit **Run the swarm**. It takes 30 seconds. Running something live is worth a lot — it proves it's real.

Point at the picture:

"Red stars are where the swarm put the access points. The orange lines are the best-known solution moving over time. You can drag this slider back to zero and watch them converge."

Drag the iteration slider back and forth once. It looks good and it costs nothing.

1.3.4 2.4 THE MOMENT — the communication ablation (90 seconds)

This is the most important 90 seconds of the whole viva. Slow down.

"The assignment asked for a problem where one weak agent fails but many together succeed. I didn't want to just assert that, so I tested it."

Untick the box that says "Particles share their global best." It re-runs by itself — you do not need to press the button.

"I've just set c2 to zero. That's the social term in the velocity update. The thirty particles are still there. They still search. They still remember their own best. They spend exactly the same 3,030 evaluations. The only thing I've taken away is that they can no longer tell each other anything."

Point at the two numbers on screen:

"The swarm scores 87.2. Random search scores 87.1. They are the same number.

Thirty searchers who don't communicate are worth no more than throwing 3,030 darts at the wall."

Note on the exact digits. The UI runs ONE seed, so what's on screen will be close to but not identical to the report, which averages 15 or 30 seeds. On screen you'll see roughly 87.2 vs 87.1; in the report it's **87.38 vs 87.43**. If he spots the difference, that's a gift — say: *"The UI is a single run, the report is a mean over 30 seeds. Over 30 seeds it's 87.38 against random's 87.43 — if anything the gap is even smaller than what you're seeing."*

Tick the box back on. It re-runs by itself again.

"Put one single number back — the global best — and the same thirty agents, at exactly the same cost, jump to 89.9. And they now connect every room in the hall, where random search leaves one room stranded.

So it isn't the population that's doing the work. It's the communication. That's the ant-in-a-bowl point from your briefing, and that's the experiment that proves it."

If he only remembers one thing from your viva, this is it.

1.3.5 2.5 If he asks "is that a fair comparison?"

"The budget is identical in every single bar — 3,030 fitness evaluations. And the code asserts it at runtime, it doesn't just trust me: `assert problem.n_fitness_calls == cfg.n_evals`. If any variant used one evaluation more than another, the program would crash."

1.3.6 2.6 Your headline table (if he wants numbers)

	score	rooms connected
PSO	89.9	100%
Random search	87.4	97.5%
Grid search	89.5	100%

"PSO beats random by 2.47 points, and it's significant — Wilcoxon signed-rank test over 15 seeds, p is 3 times 10 to the minus 5."

1.4 Part 3 — PART B, decision making (about 5 minutes)

1.4.1 3.1 Set it up (45 seconds, no maths)

"Same hall. There's a water tank on the roof, filled by an electric pump. Three things make this hard.

One — load-shedding. The power dies, and it dies worst in the evening, exactly when everyone wants a shower.

Two — there's a diesel generator, but the hall only buys two hours of diesel a day. Waste it at 7 in the morning and you have nothing at 9 at night.

Three — demand spikes, morning and evening.

So every hour you decide: do nothing, pump on grid power, or burn diesel. And the trick is you have to fill the tank BEFORE the power goes — because once it's gone, it's too late."

1.4.2 3.2 Prove the problem is worth solving (30 seconds)

He will wonder whether a simple rule would do. Answer it before he asks.

"First thing I checked was whether this actually needs planning at all, or whether a simple rule would do.

The greedy policy — that's value iteration with gamma set to zero, so it only cares about this hour — loses by 121 percent.

And even the best hand-tuned threshold rule, the one a real caretaker would use, where I grid-searched both thresholds to give it its best shot — still loses 14 and a half percent.

So the lookahead is doing real work. This isn't a fake problem."

1.4.3 3.3 Show the policy map (60 seconds)

Open `rl_policy_maps.png`.

First, tell him how to read it, because it is not obvious.

"This isn't a timeline. Nothing happens left to right. It's a lookup table.

Pick an hour along the bottom. Pick a tank level up the side. The colour tells you what to do in that exact situation. Grey is do nothing, blue is pump on grid power, red is burn diesel.

And the left and right panels are two different worlds, both of which exist at every hour. Left is 'suppose the power is ON right now'. Right is 'suppose the power is OUT right now'.

The shaded band is just the risky hours — 5 to 10pm, when demand peaks and outages are most likely. It does NOT mean the power is off. That's what the left/right split is for."

That last sentence matters. Without it, the blue inside the shaded band on the left panel looks like a contradiction — "you said the power is out, why is it pumping on the grid?" It isn't a contradiction. It's the cleverest thing in the figure, and you should say so:

"Look at the blue inside the shaded band on the left. That's the agent saying: it's 8pm, the power could go at any moment — but right now it's still on, so pump HARD while I still can. It even tops up a nearly-full tank, which it never bothers doing at 3am. That's it grabbing cheap electricity before it disappears."

Now point at the **top-right** panel.

"Hours across the bottom, tank level up the side. Red means burn diesel.

Watch the red region GROW as you go into the shaded evening window. At 7 in the morning with a low tank, it does nothing. At 7 in the evening, with exactly the same tank level, it burns diesel — because it knows the outage is coming and it knows the outage will be long.

That's the agent anticipating. That's the whole point."

If he wants it sharper, you have a witness state:

"I can point at one state. At 2pm, with the tank at 4 out of 10, pumping costs 0.97 MORE than doing nothing that hour. The optimal policy pumps anyway, because the action is worth plus 4.69 overall. So more than 5 points of that come purely from the future. The agent takes a certain loss now to hold water it'll need in four hours."

Then point at the **bottom row**:

"That's Q-learning. Same shape, but speckled and messy. That's what 'hasn't seen enough data yet' looks like."

1.4.4 3.4 THE FINDING — and it went against you (90 seconds)

Lead with the fact that you were wrong. It is the strongest thing you have.

Open `rl_model_mismatch.png`.

"Here's where the assignment actually asks 'which algorithm, and when'.

Dhaka publishes a load-shedding schedule and it's famously optimistic. So I gave Value Iteration a deliberately WRONG model — I told it outages are rarer than they really are — and then scored the policy it produced in the REAL hall.

I fully expected Q-learning to win here. It does not.

A planner whose model is wrong by a factor of four still beats Q-learning after 720,000 steps — that's 82 simulated years of running the hall.

And then I added the one baseline that could have destroyed my whole thesis. I took the exact same samples Q-learning had already seen, counted them into a model, and planned on that. That beat Q-learning by a factor of ten. Same data.

So the honest answer to 'which one, when' is: if you can write the transition structure down at all, write it down and plan. A roughly-right model is worth more than 82 years of experience. Model-free is what you reach for when you CAN'T write the model down — not when the model is merely wrong."

Pause here. Let it land. This is a negative result about your own idea, reported honestly, and it is the most impressive thing in the submission.

1.4.5 3.45 The order to show the Part B graphs

You have five. **Show three. Keep two in reserve.**

#	figure	what it is for	when
1	<code>rl_policy_maps.png</code>	what the agent <i>learned</i>	always
2	<code>rl_learning_curves.png</code>	why Q-learning is slow	always
3	<code>rl_model_mismatch.png</code>	the finding	always
4	<code>rl_vi_convergence.png</code>	the theory checks out	only if he asks
5	<code>rl_hyperparams.png</code>	which knobs matter	only if he asks

1.4.6 The middle one — why Q-learning "fails"

Open `rl_learning_curves.png`. **Red is Q-learning. Blue is a model learned from Q-learning's own samples.** Both lines are fed *exactly the same data*.

Say the correction first, because it makes you sound precise rather than defensive:

"First — it doesn't actually fail. It gets to 15% regret, which beats the greedy policy at 121% and random at 326%. It's sample-INEFFICIENT, not broken."

Then the argument, which is airtight:

"Now, why is it so slow? The obvious answer would be 'not enough data'. But look at the blue line. That's a model I built from the EXACT SAME 720,000 samples, and it gets to 1.4%. If the problem were not enough data, blue would have failed too. It didn't.

So the data was plenty. Q-learning wasted it.

Here's how. A Q-learning update takes one sample, nudges ONE cell of the table a little bit, and then throws the sample away forever. There are 3,432 cells.

A learned model stores that same sample as a count. And then value iteration re-reads that count in every one of its 2,273 sweeps. So one sample gets used thousands of times, and its consequences spread across the whole table.

It's the difference between a student who does a practice problem, scribbles the answer on a sticky note and bins the problem — and a student who writes every problem in a notebook and then re-reads the notebook a thousand times working out all the knock-on consequences. Same problems seen. Completely different amount learned."

Point at the arrow on the right of the figure: **11x apart, on identical data.**

If he pushes for the deeper reason, you have two more:

"And gamma makes it worse. Value has to crawl backwards one hop per update. Pumping at 2pm only pays off at 8pm — that's six hops, and each hop needs its own visits. At $\gamma = 0.99$ the horizon is 100 hours, so news travels very slowly.

It's also a theorem, not just my observation. Li et al. 2024 prove vanilla tabular Q-learning is stuck at $1/(1 - \gamma)^4$ to the fourth, while model-based needs $1/(1 - \gamma)^3$ cubed. At $\gamma = 0.99$ that missing factor is a hundred. My Q-learner isn't slow because I coded it badly. It's slow because it mathematically has to be."

The two you keep in reserve

- `rl_vi_convergence.png` — if he asks "how do you know value iteration converged?"
"The Bellman operator is a γ -contraction, so the error must shrink by a factor of γ every sweep. My measured rate is 0.9900. γ is 0.99. That's the theory, confirmed to four decimal places."
- `rl_hyperparams.png` — if he asks about tuning.
"Two things mattered enormously and one didn't. Turning off exploring starts triples the regret — 77% against 25%. A constant learning rate stalls at 50% where the Robbins-Monro schedule reaches 25%, exactly as Even-Dar and Mansour say it must. And epsilon? I tried 0.01, 0.05, 0.2 — barely any difference, inside the seed noise. So I report that too."

That last line is worth saying deliberately. He explicitly asked for honest null results.

1.4.7 3.5 If he asks "so why did you bother with Q-learning?"

"Because that's the answer the experiment gives, and I'd rather report it than the story I went looking for. My problem has 1,584 states and I CAN write the model down — so it's an honest advertisement for Value Iteration and a poor one for Q-learning. Knowing why is the point.

And I'll go further than you were going to: my Q-learner also loses to that two-parameter threshold rule, which does no learning at all. I'd rather say that myself than have you find it."

1.5 Part 4 — What to say about the papers

He asked you to read papers. Here is the honest, strong answer.

1.5.1 The one-liner if he just asks in passing

"Two papers actually changed the project — not just got cited. One corrected a claim I'd already written down, and one explained a result I couldn't explain."

Then give the two.

1.5.2 Paper 1 — Kennedy & Mendes (2002), swarm topology

"My first ablation was just on/off — either the particles share, or they don't. Then I read Kennedy and Mendes, who compare seventy different swarm communication structures. Their finding is that more connectivity converges FASTER but doesn't find BETTER answers.

So I implemented their ring topology, where each particle only hears its two neighbours instead of everybody. And my first write-up said the ring was better.

Then a unit test failed on different seeds, and I tested it properly on 30 paired seeds. The mean difference is NOT significant — Wilcoxon p equals 0.92. But the VARIANCE is — the ring is six times steadier, p equals 6 times 10 to the minus 6. And the fully-connected swarm gives up at iteration 84 while the ring is still improving at 98.

So the ring doesn't find a better answer. It finds a consistent one. Which is exactly what the paper says, and I had it wrong.

And one more thing — the paper does NOT actually recommend the ring. It ranks it 64th out of 70 and recommends a von Neumann lattice instead. I haven't tried that. That's the obvious next experiment."

That last paragraph is gold. It proves you read past the abstract.

1.5.3 Paper 2 — Agarwal et al. (2020) and Li et al. (2024), sample complexity

"I had the measurement — a model learned from Q-learning's own samples beats Q-learning by ten times on identical data — but I had no explanation for WHY.

The explanation turns out to be a proved theorem. Certainty-equivalence planning is minimax optimal, at 1 over (1 minus gamma) cubed. But vanilla tabular Q-learning is provably stuck at 1 over (1 minus gamma) to the FOURTH.

That's a missing factor of 1 over (1 minus gamma). I picked gamma equals 0.99. So that factor is a HUNDRED.

My Q-learner isn't slow because I coded it badly. It's slow because there's a proved sample-complexity gap, and my discount factor makes it a hundred times worse."

And then the bit that will genuinely impress him:

"I nearly cited the wrong paper for this. I was going to cite Azar 2013, and when I actually checked what it proves, it doesn't support my claim at all — its lower bound is information-theoretic and it binds model-free methods equally. So citing it would have been wrong. That's in my notes as a lesson about reading past the abstract."

1.5.4 The other papers, one line each

- **Shi & Eberhart 1998** — *"That's the inertia weight, w decaying from 0.9 to 0.4. That IS my exploration-to-exploitation mechanism, straight from the paper."*
- **Even-Dar & Mansour 2003** — *"They say a constant learning rate can't converge to Q-star, only to a neighbourhood. I didn't take it on faith — I put both in the sweep. The constant alpha stalls at 50% regret where the polynomial one reaches 25%. Their prediction, tested on my problem."*
- **No Free Lunch (Wolpert & Macready 1997)** — *"That's WHY there's an ablation at all. NFL says you can't claim 'PSO is good' — only 'PSO is good on this landscape, and here's the evidence.' So I built the experiment that could have killed my own claim."*

1.5.5 The honest bit — say it, don't hide it

"I'll be straight with you — I also have a twelve-application literature review, and those papers didn't change a single design decision. They're context, not input. If I did it again I'd mine them for encoding tricks."

Volunteering a weakness he hasn't found is worth more than hiding it.

1.6 Part 5 — If he asks about the equations

He probably will. Keep the answers short. **Don't recite the formula — say what it means, then offer the formula.**

1.6.1 "Explain your fitness function."

"It's two things subtracted. The first part is what percentage of STUDENTS have signal — weighted by how many people live in each room, not just how many rooms. The second part is a fine for putting two access points too close together — it's zero when they're far enough apart, and it grows quadratically when they're not.

So the swarm learns to spread them out without me ever telling it to."

1.6.2 "Why the sigmoid / why not just yes-or-no coverage?"

"Because a yes/no gives the optimiser nothing to follow. A room is either in or out, and nudging an access point 10 centimetres changes nothing until it suddenly flips.

The sigmoid rewards MARGIN — a room with a strong signal scores better than one that barely scrapes past the threshold. That gives the swarm a sense of warmer and colder."

1.6.3 "What does gamma actually do?"

"It's how far ahead the agent can see. One over one-minus-gamma. I used 0.99, so that's 100 hours — comfortably more than the 24-hour cycle the whole problem depends on.

If I'd used 0.9 it would only see 10 hours ahead and it would never see the evening outage coming from lunchtime."

1.6.4 "Explain the Bellman equation."

Say it as a sentence, not a formula.

"The best I can do from here equals: pick the action that maximises what I get right now, plus how good the place I land in is, discounted.

It looks circular because V-star is defined in terms of V-star. It is. But if you guess it, plug the guess into the right-hand side, and take the answer as your new guess — you get closer every time. That's value iteration. And the error shrinks by a factor of gamma every sweep, which is why my measured contraction rate is 0.9900 — that's gamma, to four decimal places."

1.6.5 "Explain the Q-learning update."

"I had a guess. Reality gave me a number. Nudge the guess towards reality, a bit.

The bracket is the surprise — what actually happened minus what I expected. If I was right, it's zero and nothing changes.

And notice what's NOT in that formula: there's no P. No probabilities anywhere. It only needs the state, the action, the reward that happened, and where it landed. That's what model-free means."

1.6.6 "Why is $R(s,a)$ an expectation?"

This is the sharpest question he can ask, and you have the sharpest answer.

"Because demand is random. Some hours two students shower, some hours five. So the reward is random too, and $R(s,a)$ is the AVERAGE reward.

And that's exactly the model-based/model-free line. Value Iteration works with the average, because it has the probability table and can compute it. Q-learning only ever sees the ONE actual reward that actually happened on that particular hour. It never sees the average, because it never sees the probabilities.

That difference IS the whole assignment, written in one line."

1.6.7 If you don't know

"I don't know — let me look."

Then open report/main.pdf and find it. **Looking something up is not a failure. Inventing a number is.**

1.7 Part 6 — Own your weaknesses before he finds them

Volunteer these. Each one makes you look stronger, not weaker.

- *"My load-shedding model is memoryless — a two-state Markov chain. Real Dhaka load-shedding is SCHEDULED, so an outage that's already run four hours is more likely to end than one that just started. My state can't represent that. If I did it again I'd add time-since-outage to the state."*
 - *"My Q-learner loses to a threshold rule that does no learning at all. I'd rather say that than have you extract it."*
 - *"I chose the discounted criterion, not average reward. For a continuing operations problem, average cost per hour is arguably more natural. I say so in the report."*
 - *"Grid search actually BEAT my PSO at first — because my rooms were on a perfect grid, so I'd handed the discrete method the answer. I added jitter so the optimum sits off the lattice. That was my mistake and it's in the report."*
-

1.8 Part 7 — If something breaks

- **UI won't start** → all the figures are already in results/plots/. Show the PNGs.
 - **He wants a number you don't have** → report/main.pdf. Open it and find it, out loud.
 - **He asks something you can't answer** → *"I don't know. Here's how I'd find out."* Then say how.
-

1.9 THE CHEAT CARD — print this, keep it in front of you

Part A, one sentence:

One particle scores 76.8. Thirty that can't talk score 87.4 — the same as random search (87.4). Thirty that share ONE number score 89.9. It's not the population, it's the communication.

Why a swarm:

Wall count is an integer, so the objective has a STEP in it. No gradient. No calculus.

Part B, one sentence:

A model that's wrong by 4× still beats Q-learning after 82 simulated years. And a model learned from Q-learning's own samples beats it tenfold. Model-free is for when you CAN'T write the model down — not when it's merely wrong.

Does it need lookahead:

Greedy loses 121%. Best tuned threshold rule loses 14.5%. Yes.

Papers:

Kennedy & Mendes corrected me — the ring is steadier (6×), not better ($p=0.92$). Li et al. explained me — Q-learning is provably stuck a factor of $1/(1-\gamma)$ behind. At $\gamma=0.99$ that's 100×.

The numbers:

Part A		Part B	
PSO	89.9	VI optimal	0%
Random	87.4	VI, 4× wrong model	2.7%
No communication	87.4	Learned model (CE)	1.4%
One particle	76.8	Q-learning (720k steps)	15.1%
Grid	89.5	Threshold rule	14.5%
Wilcoxon p	3e-05	Myopic ($\gamma=0$)	121%

If stuck: *"I don't know — let me look."* Then open the report.